

Optimization Problem - City Plan (Group B)

FEUP, MECD - Artificial Intelligence 12 March, 2026

Luana Filipa de Matos Lima
up202206845

Mariana Rocha Cristino
up202502528

Abstract—This project addresses the *City Plan* problem, a combinatorial optimization task inspired by the Google Hash Code 2018 challenge, where residential and utility buildings must be placed on a grid to maximize overall score. A Python-based framework was developed, integrating solution construction, placement validation, scoring and performance evaluation. Five optimization methods: Greedy, Hill Climbing, Simulated Annealing, Tabu Search and Genetic Algorithm, were implemented and compared on multiple benchmark datasets. Results show that trajectory-based metaheuristics, particularly Tabu Search and Simulated Annealing, achieve the best trade-off between solution quality, runtime and memory consumption, while Greedy provides a fast, reliable baseline, and Genetic Algorithms are often impractical due to computational overhead. This study highlights the importance of strategic placement and algorithm selection in complex urban layout optimization.

Index Terms—city, algorithm, optimization

I. INTRODUCTION

The *City Plan* problem is a combinatorial optimization challenge based on the Google Hash Code 2018 Final Round. Its objective is to place residential and utility buildings on a city grid to maximize the overall score of the resulting urban layout. This score depends on the accommodation capacity of residential buildings and on their access to a diverse set of utility services within a maximum walking distance, which makes the problem highly constrained and computationally complex. In this context, five optimization approaches were developed and assessed, providing a comparative analysis of different heuristic and metaheuristic techniques for efficiently tackling large-scale urban planning instances.

II. SPECIFICATION OF THE WORK PERFORMED

The work performed in this project consisted of developing a complete optimization framework for the *City Plan* problem in Python. The implemented system supports the full processing pipeline, including instance reading, city and building representation, placement validation, score computation and the collection of execution metrics. Within this framework, five optimization approaches were implemented and evaluated: Greedy, Hill Climbing, Simulated Annealing, Tabu Search and a Genetic Algorithm.

The experimental study was carried out using six benchmark datasets from the original challenge, enabling the comparison of the proposed methods across instances with different characteristics and levels of difficulty. The assessment considered

solution quality together with execution time, memory consumption and placement statistics. In addition to the optimization analysis, the project also explored a predictive component based on the experimental results, with the aim of estimating the expected performance on unseen datasets.

III. RELATED WORK

The *City Plan* problem can be viewed as combining elements of several well-known optimization families. First, it shares important similarities with multidimensional knapsack formulations, since buildings must be selected and positioned under strict spatial constraints while maximizing an objective function. In particular, the problem is not only about packing profitable items, but also about choosing combinations of buildings whose joint placement improves the final score, which makes the analogy with constrained knapsack variants especially relevant. At the same time, the scoring rule introduces a coverage component, because residential buildings benefit from access to distinct utility types within a bounded Manhattan distance. For this reason, the problem is also related to maximal covering location models, where the objective is to place facilities so as to maximize service coverage over a set of demand points. Finally, because buildings have explicit footprints that must fit in the grid without overlap, the problem also exhibits characteristics of two-dimensional packing and placement problems.

The choice of metaheuristic methods in this project is consistent with the literature on large combinatorial optimization problems. Hill Climbing and Simulated Annealing have been studied comparatively on knapsack-type problems, showing the trade-off between rapid local improvement and the ability to escape local optima. Tabu Search has also been successfully applied to quadratic knapsack variants through memory-based exploration, while guided or hybrid genetic algorithms have proved effective on multidimensional knapsack instances by combining evolutionary search with problem-specific guidance. More recent work on packing problems likewise shows that Simulated Annealing remains competitive when the search space is highly constrained and strongly dependent on geometric interactions. These studies support the use of heuristic and metaheuristic approaches for the present problem, where exact optimization would be impractical for large instances.

In addition to the academic literature, a publicly available implementation of the 2018 Google Hash Code *City Plan*

challenge was also consulted as a practical point of comparison. That project follows a divide-and-conquer strategy combined with greedy block construction, focusing on the creation of repeated neighbourhood structures with centrally placed utilities and surrounding residential buildings. Although the approach is much more specialised and less general than the metaheuristic framework developed in this work, it provided a useful external baseline for comparing solution quality and understanding how alternative design choices had been explored for the same benchmark problem.

Overall, the methods discussed in the literature are well suited to the City Plan problem, since they are able to handle large search spaces and complex placement constraints more efficiently than exact approaches. In this context, heuristic and metaheuristic methods are particularly relevant because they can produce good-quality solutions within reasonable time limits. However, they also present limitations, as their effectiveness depends strongly on parameter tuning, neighborhood structure, and the characteristics of each dataset. The approach adopted in this work follows these established directions by applying and adapting such methods to the City Plan problem, with an emphasis on practical implementation, comparative evaluation and problem-specific strategies to improve efficiency and solution quality.

IV. FORMULATION OF THE PROBLEM

A. Solution Representation

The City Plan problem is formulated as a combinatorial optimization problem on a rectangular grid of size $H \times W$. Let $B = \{0, \dots, m-1\}$ denote the set of available building projects. Each project $b \in B$ is defined by a fixed plan, a type $t_b \in \{R, U\}$, and either a residential capacity cap_b or a utility service type srv_b . A solution is represented as a variable-length set of placements

$$S = \{(b_k, r_k, c_k)\}_{k=1}^N, \quad (1)$$

where b_k is the selected building project and (r_k, c_k) gives the coordinates of the top-left corner of its plan on the city grid. This representation matches the structure of the original problem and allows the same project to be placed multiple times.

B. Rigid Constraints

A solution is feasible only if all placements satisfy the geometric and spatial restrictions of the problem. For each placement $(b, r, c) \in S$, the building must lie entirely within the grid boundaries, and its occupied cells must not overlap with those of any other placed building. If $Occ(b, r, c)$ denotes the set of occupied cells induced by placing building b at position (r, c) , then feasibility requires

$$Occ(b_i, r_i, c_i) \cap Occ(b_j, r_j, c_j) = \emptyset, \quad \forall i \neq j, \quad (2)$$

together with

$$0 \leq r \leq H - h_b, \quad 0 \leq c \leq W - w_b. \quad (3)$$

In addition, building orientations are fixed, since rotations and reflections are not allowed.

C. Evaluation Function

The objective is to maximize the total score of the city plan. Let $R(S)$ be the set of residential buildings placed in solution S . For each residential building $x \in R(S)$, let $A(x)$ be the set of distinct utility service types reachable within the maximum walking distance D , measured through the minimum Manhattan distance between occupied cells of the residential and utility buildings. The objective function is therefore

$$f(S) = \sum_{x \in R(S)} cap(x) \cdot |A(x)|. \quad (4)$$

Thus, each residential building contributes its capacity once for every distinct accessible utility type.

D. Neighbourhood, Mutation and Crossover Functions

The neighbourhood structure used by the local-search methods is defined over the placement set. The main operators are:

- **ADD**, which inserts a new valid placement, with preference given to high-capacity residential projects and compact utility types;
- **REMOVE**, which deletes an existing placement;
- **MOVE**, which relocates a building to another valid position, usually within a bounded radius;
- **CHANGE**, which replaces a building by another compatible project.

These transformations are only accepted when feasibility is preserved.

In the Genetic Algorithm, each individual is also represented as a valid placement set. *Mutation* is not modelled as a purely random perturbation, but rather as a guided modification followed by local refinement. In particular, offspring may undergo a destroy-and-repair step, in which part of the current placement set is removed and then rebuilt, or a memetic local-search phase based on **ADD**, **MOVE** and **REMOVE** operations. The **ADD** operator tends to favour high-quality residential projects and compact utility candidates, often placing them near existing buildings, whereas **MOVE** attempts a small local relocation and keeps the best position among a few nearby trials. This makes mutation both exploratory and exploitative, rather than purely random.

Crossover combines placements from two parent solutions while maintaining feasibility. Three variants were implemented. The default `spatial crossover` exchanges placements according to randomly selected spatial windows of the grid, so that each child inherits one region from one parent and is then completed with non-conflicting placements from the other parent. A `score-aware crossover` instead prioritizes placements with higher local contribution to the objective function. A `clustered crossover` builds offspring around utility-centered clusters, preserving groups of nearby placements that already interact well. After crossover, children may be further completed by guided random insertion and refined by local search. Therefore, crossover is designed not merely to mix placements, but to preserve useful spatial structure from both parents.

V. IMPLEMENTATION DETAILS

The project is organized in a modular way to separate problem representation, algorithm implementations, and utilities for evaluation and experimentation. This structure makes it easier to test different optimization algorithms and reuse core components of the problem model.

A. Development Environment

The development environment consisted of:

- Programming language: Python 3
- Operating system: Linux
- IDE: Visual Studio Code
- Libraries and Tools:
 - Standard libraries: `os`, `time`, `importlib`, `inspect`, `collections`
 - * Used for filesystem operations, dynamic imports of algorithms, queue structures, performance measurement, and warning suppression.
 - Data analysis and handling: `pandas`, `numpy`
 - * Used for reading and writing metrics, manipulating datasets, and efficient numerical computations.
 - Machine learning and regression: `sklearn`
 - * Used for optional predictive analysis of city plan scores or building placements.
 - Visualization: `matplotlib`
 - * Used to plot city layouts, score distributions, and algorithm performance.
 - Formatting: `tabulate`
 - * Used for console-friendly display of metrics tables.

B. Project File Structure

The project follows a structured directory organization to clearly separate datasets, algorithms, utilities, and experimental results.

```
CITYCRAFTER/
|-- data/
|   |-- a_example.in
|   |-- b_short_walk.in
|   |-- c_going_green.in
|   |-- d_wide_selection.in
|   |-- e_precise_fit.in
|   '-- f_different_footprints.in
|
|-- results/
|   |-- metrics.csv
|   '-- tests_metrics.csv
|
|-- src/
|   |-- algorithms/
|   |   |-- greedy.py
|   |   |-- hill_climbing.py
|   |   |-- simulated_annealing.py
|   |   |-- tabu_search.py
|   |   '-- genetic.py
|   |
|   |-- utils/
|   |   |-- metrics.py
|   |   '-- visualization.py
|   |
|   |
```

```
|   |-- buildings.py
|   |-- city.py
|   |-- coordinates.py
|   |-- read_file.py
|   |-- main.py
|   '-- Prediction_and_Results.ipynb
|
|-- tests/
|   |-- g_joinbd.in
|   |-- h_compact_balanced.in
|   |-- i_joinbd.in
|   '-- j_precise_fit.in
```

The **data** folder contains the official datasets used for experimentation. The **tests** folder contains additional datasets used for validation and debugging. The **results** folder stores experimental metrics collected during algorithm runs.

C. Data Structures

The implementation models the problem using several object-oriented classes that encapsulate the key entities of the problem: coordinates, building projects, and the city instance.

1) *Coordinates*: The `Coordinates` class represents positions in the city grid using row and column indices. It also provides utility methods such as Manhattan distance calculation and tuple conversion. This abstraction simplifies spatial operations across the codebase.

A derived class, `CityCoordinate`, extends this concept by including metadata about the cell, such as whether it contains a residential or utility building and which project it belongs to.

2) *BuildingProject*: The `BuildingProject` class represents one building project defined in the input dataset. It stores:

- the project identifier
- the building dimensions
- the building plan (grid of # and .)
- the type of building (R for residential or U for utility)
- the residential capacity or utility service type

During initialization, the class also performs **validation of the building plan** to ensure it satisfies all constraints specified in the problem statement:

- each edge of the plan contains at least one occupied cell (#)
- all occupied cells form a single connected component
- there are no holes inside the building plan

To improve performance during placement checks, the class precomputes the **relative positions of all occupied cells (#)**. When a building is placed in the city, these offsets can be quickly translated to absolute coordinates, avoiding repeated scans of the entire building grid.

3) *City*: The `City` class represents a complete instance of the optimization problem. It stores:

- grid dimensions $H \times W$
- the maximum walking distance D
- the number of building projects B
- the list of all available `BuildingProject` objects

This class is responsible for:

- storing the problem instance
- executing optimization algorithms
- computing solution scores
- collecting runtime metrics

The method `make_city` dynamically loads and executes a chosen algorithm from the `algorithms` folder. This dynamic loading approach allows all algorithms to share the same interface and makes it easy to experiment with different methods without modifying the core infrastructure.

D. Algorithm Integration

All optimization algorithms are implemented in the `algorithms` folder. The project includes implementations of several local search and metaheuristic techniques:

- 1) Greedy Algorithm (baseline)
- 2) Hill Climbing
- 3) Simulated Annealing
- 4) Tabu Search
- 5) Genetic Algorithm

Each algorithm receives a `City` instance as input and returns a solution, which is represented as a list of building placements.

To ensure flexibility, the system normalizes solution formats using helper functions that convert placements into a consistent internal representation.

E. Solution Representation in Code

A solution is represented as a list of placements, where each placement specifies:

- the building project ID
- the top-left coordinate where the building is placed

Internally, placements are stored as tuples or small objects containing (`project_id`, `row`, `column`).

When evaluating a solution, the algorithm converts these placements into absolute positions of occupied cells using the building plan offsets.

F. Score Computation

The score computation is implemented in the `City.get_score()` method. The method works in two main stages.

First, it computes the influence area of each utility building. For each occupied cell of a utility building, all cells within the Manhattan distance D are marked as being served by that utility type.

Second, for each residential building, the algorithm determines which utility types are reachable from any of its occupied cells. The score contribution of that residential building is calculated as:

$$\text{capacity} \times (\text{number of distinct reachable utility types})$$

The final score is obtained by summing the contributions of all residential buildings.

G. Performance Measurement

To evaluate algorithm performance, the system automatically records several metrics for each run. These metrics are saved in CSV format using the utility module `metrics.py`, allowing further analysis and visualization.

H. Usage

The entry point of the application is the `src/main.py` script, which features an interactive Command-Line Interface (CLI) called the City Crafter Menu. Upon execution, this menu guides the user through a simple two-step setup process. First, the user is prompted to select one of the five implemented optimization algorithms (Greedy, Hill Climbing, Simulated Annealing, Tabu Search, or Genetic Algorithm). Following this, the user chooses the specific dataset instance they wish to evaluate, ranging from 'a' to 'j'. This intuitive interface allows for seamless testing and comparison of different algorithm-dataset combinations.

```
=====
CITY CRAFTER MENU
=====

=== Choose Algorithm (or 'q' to quit) ===
1) Greedy
2) Hill Climbing
3) Simulated Annealing
4) Tabu Search
5) Genetic
Algorithm number: 1
Selected algorithm: Greedy

=== Choose Dataset or 'q' to quit ===
Available: a, b, c, d, e, f, g, h, i, j
Dataset letter (a-j): █
```

Fig. 1: City Crafter Menu

VI. APPROACH

The optimization strategy adopted in this project combines constructive heuristics with local-search and population-based metaheuristics. Instead of attempting to directly solve the problem through exhaustive search, the approach focuses on progressively improving city layouts by exploring neighbouring solutions while maintaining feasibility.

The overall pipeline follows three main stages:

- 1) Construction of an initial feasible solution using a Greedy heuristic.
- 2) Iterative improvement through neighbourhood-based optimization methods.
- 3) Exploration of larger regions of the search space through population-based recombination in the Genetic Algorithm.

This design allows the algorithms to balance exploration of new layouts with exploitation of promising configurations.

A. Initial Solution Construction

All optimization algorithms start from a solution generated by a Greedy heuristic. The purpose of this heuristic is not to produce an optimal layout, but to quickly construct a valid city layout that already contains a balanced distribution of residential and utility buildings.

This initial solution provides a structured starting point for the subsequent optimization algorithms, which then refine the layout through neighbourhood exploration. The greedy strategy itself is described in detail in Section VII.

B. Local Search Operators

To improve the initial layout, the algorithms rely on a set of local modification operators that define the neighbourhood structure of the search space. These operators allow the algorithms to explore nearby solutions by modifying the set of building placements.

Four main operators are used:

- **ADD**: inserts a new building placement into the solution. Candidate projects are selected with preference for high-capacity residential buildings and compact utilities, and are placed in valid grid positions.
- **REMOVE**: deletes an existing placement from the solution. This operator allows the algorithm to eliminate buildings that provide little contribution to the overall score.
- **MOVE**: relocates a building to another valid position, typically within a limited spatial radius around the original placement. This operator is used to locally refine building positions.
- **CHANGE**: replaces a building by another project of the same type (residential or utility). This enables the algorithm to explore alternative building configurations while keeping the spatial structure of the layout.

C. Genetic Operators

The Genetic Algorithm introduces additional mechanisms for exploring the search space through the recombination of solutions. Each individual in the population represents a complete city layout encoded as a set of building placements.

Crossover operators combine placements from two parent solutions to generate new candidate layouts. After crossover, offspring solutions may undergo mutation and local refinement. Mutation is implemented through guided perturbations, such as partial removal of placements, followed by reconstruction of the affected region. This destroy-and-repair mechanism helps the algorithm escape local optima while preserving useful structural components of the layout.

VII. HEURISTIC OPTIMIZATION ALGORITHMS

The optimization framework includes five algorithms designed to explore the solution space using different search strategies. These methods range from a constructive heuristic used to generate initial solutions to several local-search and population-based metaheuristics that progressively improve the city layout. Each algorithm relies on the neighbourhood operators defined previously, while applying different exploration and acceptance mechanisms.

A. Greedy

The Greedy algorithm was implemented as a constructive heuristic aimed at rapidly generating a feasible and reasonably structured city layout. Instead of exploring the solution space iteratively, this method builds the placement set incrementally by selecting buildings and positions according to simple heuristic criteria. The resulting solution serves both as a baseline for comparison and as the initial configuration used by the subsequent optimization algorithms.

The algorithm begins by separating the available projects into residential and utility categories. Residential projects are ranked according to a quality metric that favours buildings with high accommodation capacity relative to their footprint, while utility projects are filtered so that only compact representatives of each service type are considered. This selection reduces the candidate space and prioritizes projects that are more likely to contribute positively to the objective function.

Two placement strategies are used depending on the walking distance parameter D . When the distance is small, the algorithm performs a dense sweep of the grid, placing residential buildings in regularly spaced blocks and attempting to insert nearby utilities around each residential placement. This strategy quickly produces compact neighbourhoods where services can reach nearby residents.

For larger walking distances, a hub-based strategy is adopted. In this case, the algorithm first attempts to construct clusters of utility buildings that act as service hubs. These hubs are distributed across the grid using a regular spatial step and are built by packing compact utilities of different service types near the hub centre. After the utility hubs are created, the surrounding areas are filled with residential buildings whose positions are selected based on their potential to reach multiple utility services.

To guide this process efficiently, the algorithm relies on several heuristics, including prioritization of high-capacity residential projects, preference for compact utilities, and local evaluation of the number of reachable service types before inserting a residential placement. Candidate positions are sampled around hubs and evaluated according to an estimated contribution score that combines residential capacity, number of reachable utilities, and building footprint.

The greedy procedure continues placing buildings until the grid becomes saturated or a predefined runtime limit is reached. Although the resulting layout is not guaranteed to be optimal, it typically provides a well-distributed structure of residential and utility buildings, making it a strong starting point for the local-search and metaheuristic algorithm used later in the optimization process.

B. Hill Climbing

Hill Climbing was implemented as a deterministic local-search method aimed at progressively improving a feasible city layout through incremental modifications. The algorithm starts from an initial solution and repeatedly applies neighbourhood transformations that attempt to increase the overall score of the city plan. Unlike stochastic metaheuristics, Hill Climbing

only accepts changes that do not decrease the objective value, making the search strongly exploitative.

The algorithm begins by generating an initial placement set using the Greedy heuristic described previously. The resulting solution typically contains a large number of residential and utility buildings distributed across the grid, providing a structured starting point for the local search. These placements are then injected into the state, where auxiliary data structures are initialized to track occupied cells and utility influence areas across the grid.

At each iteration, the algorithm selects one neighbourhood operator according to predefined probabilities. The available operators include ADD, REMOVE, MOVE and CHANGE. Candidate placements are tested for feasibility before being applied, ensuring that all grid and overlap constraints remain satisfied.

Once a candidate modification is generated, the algorithm recomputes the resulting score. If the new solution improves or maintains the current score, the change is accepted; otherwise, the algorithm restores the previous configuration. This greedy acceptance rule drives the search toward locally optimal configurations while avoiding unnecessary exploration of clearly inferior layouts.

To reduce the computational cost of repeated evaluations, the implementation maintains incremental structures that track occupied grid cells and the influence of utility services on surrounding locations. These structures allow the algorithm to efficiently update the score of residential buildings after each modification without recomputing the entire layout.

The search process continues for a maximum number of iterations or until no improvement is observed for a given number of consecutive iterations, controlled by the parameter *patience*. When this threshold is reached, the algorithm terminates early to avoid wasting time exploring a stagnating region of the search space. An optional multi-restart mechanism is also included: after completing one run, the algorithm may restart the Hill Climbing process from a new greedy-generated solution, keeping the best layout found across all runs.

The main parameters controlling the behaviour of the algorithm include *max_iterations*, which limits the total number of neighbourhood explorations, *patience*, which defines how long the search continues without improvement, and *min_delta*, which determines the minimum score increase required to reset the stagnation counter. Together, these parameters balance runtime efficiency with the ability to refine promising city layouts.

C. Tabu Search

Tabu Search was implemented as an advanced local-search metaheuristic designed to overcome the limitations of basic Hill Climbing by allowing the exploration of non-improving moves while avoiding search cycles. The algorithm maintains a short-term memory structure, known as the tabu list, which temporarily forbids recently applied moves. This mechanism encourages diversification and helps the search escape from local optima.

Similar to the other optimization algorithms, the procedure begins with an initial solution generated by the Greedy heuristic. This starting configuration typically contains a well-distributed set of residential and utility buildings and provides a strong baseline from which the search can begin its refinement process. The placements generated by the greedy algorithm are inserted into the internal structures of the Tabu Search algorithm, which track occupied grid cells, utility influence regions, and residential coverage information.

At each iteration, the algorithm explores a sampled subset of the neighbourhood around the current solution. Candidate moves are generated using the same operators defined for the local-search framework: ADD, REMOVE, MOVE and CHANGE. Instead of evaluating all possible neighbours, the algorithm randomly samples a fixed number of candidate moves, defined by the *neighborhood_size* parameter, which significantly reduces computational overhead while still providing sufficient exploration of the search space.

Each candidate's move is evaluated by temporarily applying the modification and computing the resulting score. The best non-tabu move found within the sampled neighbourhood is then selected and permanently applied to the solution. Moves are represented by signatures describing the action performed, such as adding a building at a specific position or moving a building from one location to another. These signatures are stored in the tabu list for a limited number of iterations, preventing the algorithm from immediately reversing recent changes.

An aspiration criterion is also implemented to override the tabu restriction when a move produces a solution that improves the best score found so far. This mechanism ensures that high-quality solutions are never discarded due to tabu constraints, allowing the search to continue progressing toward better configurations.

To efficiently evaluate candidate moves, the algorithm maintains incremental data structures that track the influence of utility buildings across the grid and the coverage of residential buildings by nearby services. When buildings are added or removed, only the affected regions of the grid are updated, allowing the algorithm to compute score variations without recomputing the entire layout.

The search continues for a predefined number of iterations or until the global runtime limit is reached. Throughout the process, the algorithm keeps track of the best configuration encountered and returns this solution at the end of the execution. The main parameters controlling the behaviour of the algorithm include *max_iterations*, which limits the number of search steps, *tabu_tenure*, which defines the size of the tabu list, and *neighborhood_size*, which controls how many candidate moves are evaluated at each iteration.

D. Simulated Annealing

Simulated Annealing was implemented as a stochastic local-search metaheuristic designed to improve a feasible city layout while reducing the risk of getting trapped in poor

local optima. Unlike Hill Climbing, which only accepts non-decreasing moves, Simulated Annealing occasionally accepts worsening modifications according to a temperature-controlled probability. This mechanism promotes diversification during the early stages of the search and allows the algorithm to explore less obvious regions of the solution space.

As in the other optimization methods, the procedure begins with an initial solution generated by the Greedy heuristic. This starting configuration typically provides a valid and well-distributed arrangement of residential and utility buildings, which is then loaded into the internal state of the algorithm. The implementation maintains auxiliary structures to track occupied cells, utility influence regions, and residential service coverage, allowing the current score to be updated efficiently after each modification.

At each iteration, the algorithm selects one neighbourhood operator according to predefined probabilities: ADD, REMOVE, MOVE or CHANGE. This design allows the search to explore both structural and local variations of the current solution.

Each candidate move is evaluated by temporarily applying the corresponding modification and measuring the resulting score variation. If the new solution improves the current score, the move is always accepted. Otherwise, it may still be accepted with probability $e^{\frac{\Delta}{T}}$, where Δ is the score variation and T is the current temperature. This acceptance rule allows the algorithm to escape local optima by occasionally retaining non-improving moves, especially when the temperature is still high. The temperature is reduced gradually according to a geometric cooling schedule controlled by the parameters *initial temperature*, *cooling rate* and *min temperature*. A limited reheating mechanism is also included, through the parameters *reheats* and *reheat factor*, in order to reintroduce diversification when the search becomes too conservative.

This starting configuration typically provides a valid and well-distributed arrangement of residential and utility buildings, which is then loaded into the internal state of the algorithm. The implementation maintains auxiliary structures to track occupied cells, utility influence regions and residential service coverage, allowing the current score to be updated efficiently after each modification.

Additional parameters such as *max_iterations* and *max_runtime_s* control the scale and frequency of the neighbourhood transformations. The search proceeds until the maximum number of iterations is reached, the runtime budget is exhausted, or the temperature can no longer be meaningfully reduced. Throughout the process, the best solution encountered is stored and returned at the end of the execution.

E. Genetic Algorithm

The Genetic Algorithm was implemented as a population-based metaheuristic designed to explore larger regions of the search space through recombination of complete city layouts. Unlike the local-search methods, which progressively modify a single current solution, the Genetic Algorithm maintains a population of candidate layouts and evolves them across

successive generations. However, the final implementation does not follow a purely standard genetic design. Instead, it takes the form of a memetic algorithm, combining the classical evolutionary cycle of selection, crossover, and mutation with local improvement and problem-specific repair procedures. This hybrid structure was adopted because the City Plan problem exhibits strong spatial dependencies, meaning that the quality of a placement depends not only on the individual building itself but also on the surrounding configuration of residential and utility structures.

As in the other optimization methods, the procedure begins from a solution generated by the Greedy heuristic. Rather than constructing the initial population entirely at random, the algorithm first converts this greedy layout into one valid individual and then generates diversified variants of it. This choice provides the population with a strong initial baseline while avoiding the poor quality usually associated with fully random city configurations. Each individual represents a complete city layout together with auxiliary structures that track occupied cells, utility influence regions, residential service coverage, and the current score. These internal structures allow overlap tests and score updates to be performed incrementally, which is especially important in this algorithm because crossover, mutation, and local improvement generate a large number of intermediate modifications.

At each generation, parent individuals are selected through tournament selection, while the best layouts are preserved through elitism. In this way, the search maintains selection pressure towards higher-quality solutions without discarding the strongest individuals found so far. Crossover is one of the main components that distinguishes this implementation from a traditional genetic algorithm. Since a city layout is not naturally represented as a linear chromosome, simple one-point or uniform crossover would not preserve meaningful spatial structure. For this reason, three specialised crossover strategies were implemented. The default spatial crossover exchanges rectangular regions of the grid between parent solutions, preserving geographically coherent substructures. A score-aware crossover instead favours placements with higher estimated local contribution to the objective function, giving priority to buildings that appear to be more valuable within the layout. A clustered crossover goes one step further by preserving groups of buildings organised around strong utility anchors, thereby attempting to transfer effective local service structures from one solution to another. These operators make crossover more informed and more suitable for the spatial nature of the problem than a basic gene-by-gene recombination scheme.

Mutation is also designed to be less conservative and more exploratory than in a standard genetic algorithm. Instead of applying only small random perturbations, the implementation combines mutation with domain-specific reconstruction and local search. One mechanism is destroy-and-repair, in which a random region of the grid is partially cleared and then rebuilt through a guided filling procedure. This allows the algorithm to modify an entire local area of the city rather

than changing only isolated placements, which is useful when a poor spatial arrangement must be restructured more substantially. In addition, newly generated offspring may undergo a memetic local-search phase based on the same neighbourhood operators used by the local-search methods, namely ADD, MOVE, and REMOVE. This means that offspring are not only created through recombination, but are also refined before being reinserted into the population. As a result, the algorithm combines exploration through evolutionary variation with intensification through local improvement, which is precisely what characterises a memetic approach.

A further distinction from a basic genetic algorithm is the use of an adaptive population size. Rather than keeping the population fixed throughout the whole execution, the algorithm adjusts it according to the observed search progress. When several generations pass without improvement, the population may be expanded in order to inject additional diversity and reduce premature convergence. Conversely, when the search starts improving again, the population may shrink towards its base size, concentrating the effort on intensification and limiting computational overhead.

Therefore, the proposed method extends the traditional genetic framework with informed initialisation, incremental evaluation, specialised crossover operators, destroy-and-repair mutation, memetic local refinement and adaptive population control. These additions make it considerably better suited to the large, structured and highly interdependent search space of the City Plan problem than a conventional genetic algorithm.

VIII. PREDICTION

As an additional component of the project, a predictive analysis was developed to estimate the expected score of each optimization algorithm on unseen datasets. The aim of this stage was not to replace direct experimentation, but rather to explore whether the structural characteristics of a city instance could be used to anticipate algorithmic performance and support the selection of a suitable optimization method.

The predictive model was trained using the experimental results collected from the six official benchmark datasets. For each dataset, a set of descriptive features was extracted directly from the input files, including grid dimensions, total area, maximum walking distance, number of projects, number of residential and utility buildings, average and maximum building areas, average residential capacity, number of utility service types, and density-related measures derived from the building footprints. These dataset-level features were then combined with the algorithm identity in order to predict the final score obtained by each method.

A Random Forest Regressor was selected as a surrogate model because it can capture non-linear relationships between dataset characteristics and score outcomes without requiring strong assumptions about the data distribution. After one-hot encoding the algorithm variable, the model was trained to predict the score associated with each dataset–algorithm pair. On the held-out validation split, it achieved a mean absolute error of 114487.65 and an R^2 value of 0.6657, indicating a

moderate but meaningful predictive capacity. In addition, when the predictions for the unseen test datasets were compared with the real scores, the mean absolute percentage error was 7.42%, which suggests that the model was generally able to estimate the performance of the algorithms with reasonable accuracy.

The analysis of the unseen datasets showed that the model consistently identified Tabu Search as the most promising algorithm for all four new test instances. Once the real scores became available, this recommendation proved to be correct for three of the four cases, while the remaining case was won by Simulated Annealing. The individual prediction errors were mostly moderate, although larger deviations appeared for some Simulated Annealing and Tabu Search cases, especially on datasets with more irregular behaviour. Even so, the overall results indicate that a lightweight predictive model can provide useful guidance about expected algorithm performance before the full optimization process is executed.

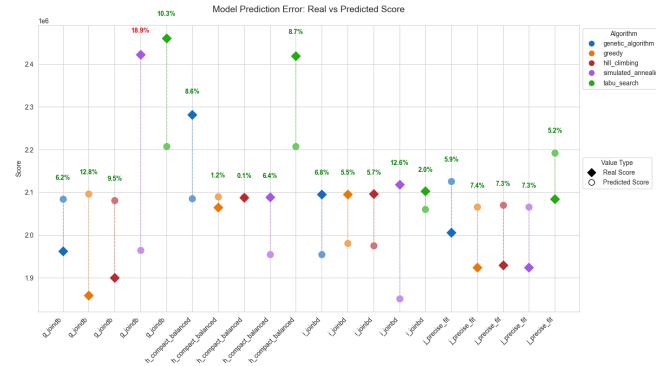


Fig. 2: Model Prediction Error

Although this predictive component was developed as an exploratory extension rather than as a core part of the optimization framework, it adds an interesting practical dimension to the project. In particular, it suggests that dataset features can be used not only for problem analysis, but also for algorithm recommendation, thereby complementing the experimental comparison with a data-driven estimate of which method is likely to perform best on a new instance.

IX. EXPERIMENTAL RESULTS

A. Execution Time by Algorithm and Dataset

This visualization shows the execution time of the five optimization algorithms across the five benchmark datasets. Bars represent the average runtime in seconds, while error bars indicate variability across multiple executions.

The *Greedy* algorithm is consistently the fastest across almost all datasets, with negligible error bars, reflecting highly deterministic behaviour. This efficiency stems from its constructive nature, generating solutions without extensive exploration of the search space.

Hill Climbing exhibits long runtimes with large variability, particularly in *c_going_green*, indicating sensitivity to stochastic factors: some runs converge quickly while others require significantly more time.

Simulated Annealing demonstrates a balanced runtime profile, consistently faster than Hill Climbing and Tabu Search, with moderate variability, representing a good compromise between exploration and efficiency.

The *Genetic Algorithm* stands out differently. Even though it was allocated a longer maximum runtime than other algorithms, it does not complete within this time limit. This behaviour reflects the inherent complexity of population-based methods: maintaining and evolving large candidate populations is highly time-consuming, and the algorithm may fail to converge fully under practical computational constraints.

Dataset characteristics also influence runtime. *c_going_green* is the most demanding, while *b_short_walk* is relatively easy for Greedy, Simulated Annealing, and Tabu Search. Hill Climbing and Genetic Algorithm still require substantial time, highlighting that search-based methods can struggle even with simpler datasets.

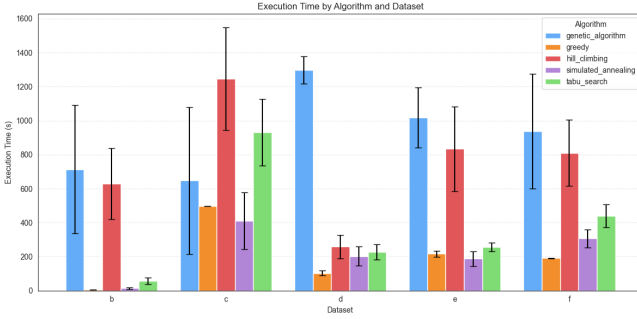


Fig. 3: Execution Time by Algorithm and Dataset

B. Peak Memory Usage by Algorithm and Dataset

This figure displays the peak memory usage (MB) for each algorithm across datasets on a logarithmic scale.

The *Genetic Algorithm* consumes the most memory, often approaching 10^4 MB (10 GB), with high variability. Even within extended time limits, its memory footprint remains significant due to storing large populations and all candidate solutions across generations.

Greedy uses minimal memory (10^1 MB), consistent across datasets, due to its sequential, non-exploratory nature.

Hill Climbing, *Simulated Annealing*, and *Tabu Search* fall between 10^2 and 10^3 MB. Tabu Search and Simulated Annealing occasionally require slightly more memory than Hill Climbing due to auxiliary structures like the tabu list or probabilistic acceptance mechanisms.

C. Score by Algorithm and Dataset

This visualization compares solution quality across algorithms. Higher scores indicate better solutions.

Tabu Search consistently achieves the highest scores, particularly in *b_short_walk*, *e_precise_fit*, and *f_different_footprints*, effectively escaping local optima via its memory-based search.

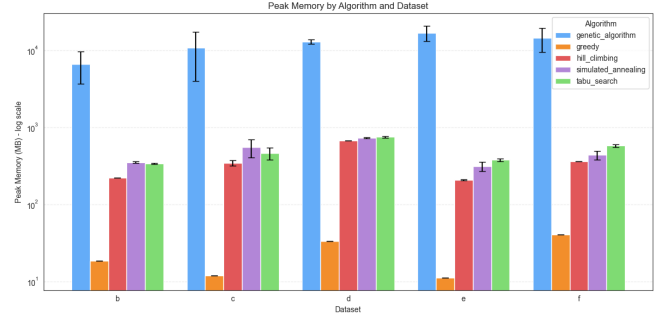


Fig. 4: Peak Memory by Algorithm and Dataset

Simulated Annealing excels in challenging datasets like *c_going_green*, where probabilistic acceptance of worsening moves promotes diverse exploration.

Greedy remains fast and memory-efficient but typically yields lower scores, except in *f_different_footprints*. Minimal error bars reflect its deterministic behaviour.

Genetic Algorithm, despite extended time limits, shows inconsistent and often incomplete performance. Its scores do not reliably surpass trajectory-based metaheuristics, and large variability indicates sensitivity to stochastic factors and the potential for non-convergence within practical timeframes.

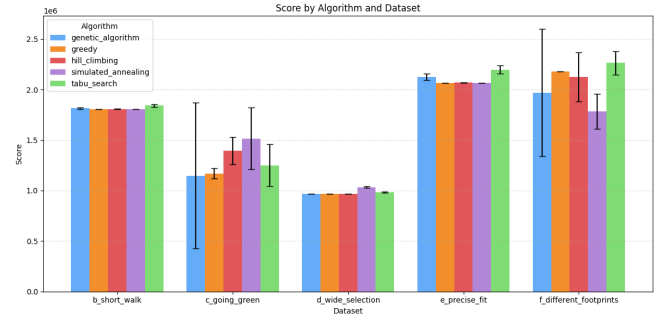


Fig. 5: Score by Algorithm and Dataset

D. Best Algorithm by Dataset

This figure highlights the highest-scoring algorithm per dataset. Notably, *Greedy* and *Hill Climbing* do not appear as top performers, reflecting their inability to consistently reach optimal solutions under the given constraints. The *Genetic Algorithm* also does not appear, not because of low potential, but because even with extended runtime it did not complete execution for most datasets, often not reaching even the halfway point of the optimization process.

The best results are therefore exclusively achieved by trajectory-based metaheuristics: *Tabu Search* dominates three datasets (*b_short_walk*, *e_precise_fit*, *f_different_footprints*), while *Simulated Annealing* performs best in *c_going_green* and *d_wide_selection*. This confirms the effectiveness of their exploration mechanisms in complex search spaces.

and underscores that, for practical purposes, population-based methods like Genetic Algorithm may be infeasible under realistic time constraints.

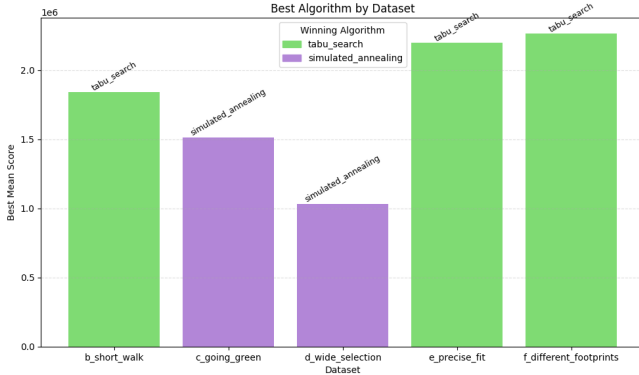


Fig. 6: Best Algorithm by Dataset

E. Maximum Delta Relative to Greedy by Dataset and Algorithm

This visualization presents maximum score improvements relative to Greedy. Significant gains occur in `c_going_green` and `f_different_footprints`, justifying advanced metaheuristics. Datasets like `b_short_walk` and `e_precise_fit` show marginal improvements, where Greedy remains sufficient. Genetic Algorithm sometimes fails to improve over Greedy, emphasizing instability and incomplete convergence, whereas Tabu Search consistently delivers substantial improvements.

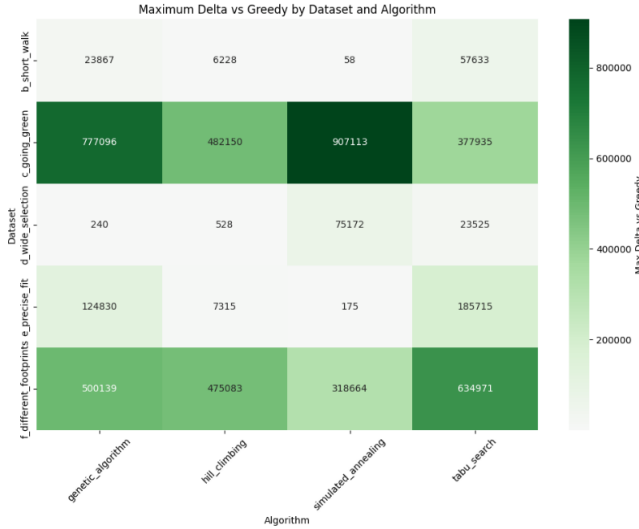


Fig. 7: Maximum Delta vs Greedy by Dataset and Algorithm

F. Algorithm Score and Time by Scenario

Hyperparameter scenarios affect both Score and runtime:

Hill Climbing: Scenario 1 balances runtime (600s) and score. Scenario 2, with excessive iterations, increases runtime

(1000s) without score gains, illustrating wasted computation in local optima.

Simulated Annealing: Scenario 3 increases iterations and reheats, resulting in longer runtime (> 300s) but superior scores, demonstrating the benefit of intensive exploration.

Tabu Search: Scores remain stable (1.7M) across scenarios; minor adjustments improve scores marginally but increase runtime.

Genetic Algorithm: Clustered and Score_aware crossovers perform best, yet even with extended runtime, the algorithm may not complete, confirming that larger populations and longer generations increase computational cost without guaranteeing convergence.

Hyperparameter tuning significantly affects performance. Trajectory-based algorithms benefit from configurations promoting intensive exploration. Population-based algorithms like Genetic Algorithm require careful adjustment; overly large populations or aggressive mutations increase runtime and may prevent convergence, while focused configurations ensure consistent high-quality solutions.

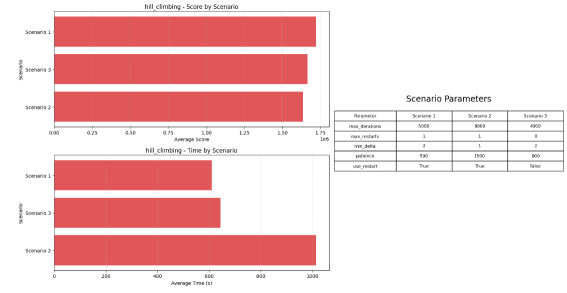


Fig. 8: Hill Climbing Score and Time by Scenario

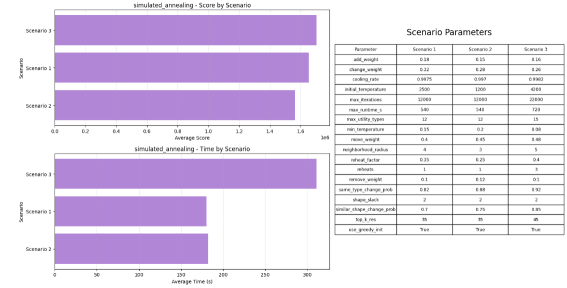


Fig. 9: Simulated Annealing Score and Time by Scenario

X. DASHBOARD: TRADE-OFFS, METRICS, AND RECOMMENDATIONS

This final figure serves as an **Executive Dashboard**, summarizing all analyses and addressing the question "Which algorithm should I use?".

A. Upper-Left Quadrant: Placements vs Score

The scatter plot shows a surprising insight: more placements do not necessarily lead to higher scores. Points cluster by dataset (shape), not algorithm (color), indicating that dataset

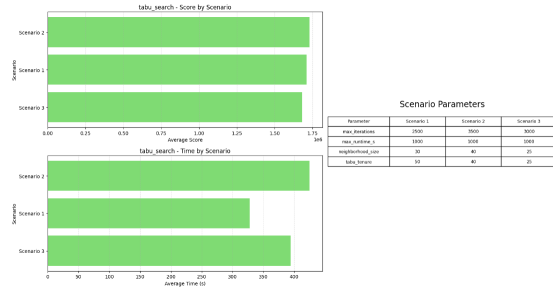


Fig. 10: Tabu Search Score and Time by Scenario

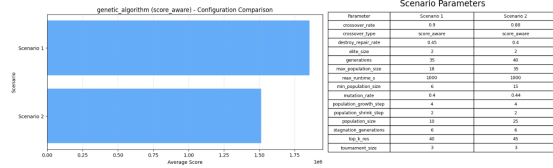


Fig. 11: Genetic (Score Aware) Score and Time by Scenario

topology constraints achievable scores regardless of algorithmic sophistication.

- Dataset *f*: Nearly 250,000 placements on average, but among the lowest scores (1M), due to its 180 projects and 19 service types, creating conflicts and inefficiencies.
- Dataset *b* and *e*: Require far fewer placements (25,000-50,000) yet achieve maximum scores (> 2M), reflecting simpler configurations (fewer projects and service types) that allow rapid optimization.

B. Upper-Right Quadrant: Dataset Characteristics Table

The table explains the observed behaviours:

- High efficiency of Dataset *e*: Only 20 projects and 1 service type; straightforward constraints allow algorithms to maximize score with minimal placements.
- Low efficiency of Dataset *f*: 180 projects and 19 service types; complex spatial and service conflicts force many low-value placements, limiting overall score.

C. Lower-Left Quadrant: Pareto Frontier (Score vs Time)

The dashed line represents the Pareto Frontier, i.e., the set of best trade-offs between runtime and score:

- **Efficient algorithms (on the line):** *Greedy* dominates the left (instantaneous, low score), *Simulated Annealing* occupies the middle, *Tabu Search* dominates the right (longer runtime, highest scores).
- **Inefficient algorithms (below the line):** *Genetic Algorithm* (blue) and *Hill Climbing* (red) mostly lie below the frontier. The Genetic Algorithm did not even complete execution for many datasets within the allowed runtime, making it infeasible in practice. These methods spend more time without achieving superior scores compared to Tabu Search or Simulated Annealing.

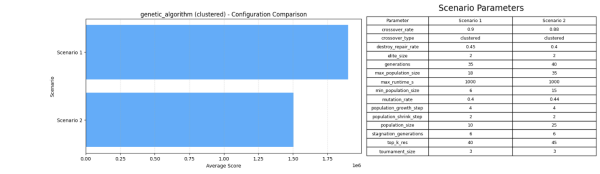


Fig. 12: Genetic (Clustered) Score and Time by Scenario

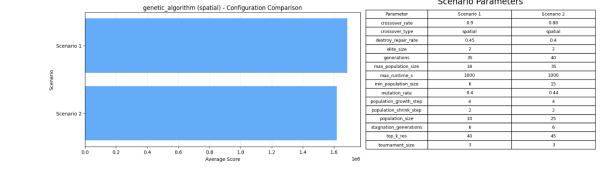


Fig. 13: Genetic (Spatial) Score and Time by Scenario

D. Lower-Right Quadrant: Algorithm Profile Heatmap

The normalized heatmap (0-1) provides a comprehensive comparison:

- Genetic Algorithm: Peaks at 1.00 in Score due to sporadic high outcomes, but scores 0.00 in all other metrics; slow, memory-intensive, inconsistent, and often incomplete, it's the least practical choice.
- Greedy: Perfect in Time and Memory (1.00) but minimal Score (0.00), reliable but suboptimal.
- Simulated Annealing: It was very consistent (1.00), fast (0.97), memory-efficient (0.96), moderate Score; excellent for reliable solution quality.
- Tabu Search: Most balanced; maximizes placements (1.00), efficient in memory (0.96), strong normalized Score (0.53); ideal for high-quality, practical solutions.

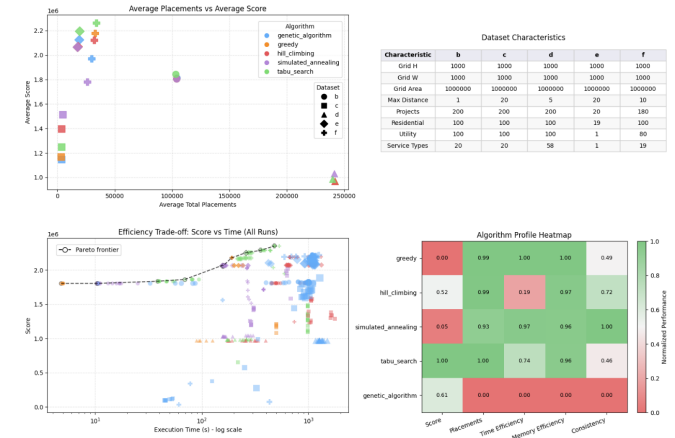


Fig. 14: Dashboard

XI. CONCLUSION

This project addressed the *City Plan* optimization problem by developing a comprehensive framework that models the problem, generates feasible layouts, and improves them using a variety of heuristics and metaheuristics. The implementation integrates constructive heuristics, neighbourhood-based local

search, and population-based optimization methods within a unified experimental environment.

The *Greedy* algorithm provides a fast and reliable initial solution, establishing meaningful spatial organization between residential and utility buildings. This baseline is particularly valuable when computational resources or runtime are limited, as it delivers predictable outcomes with minimal memory consumption. However, its deterministic nature limits the potential score, especially in more complex datasets like `c_going_green` and `f_different_footprints`.

Trajectory-based local search algorithms (*Hill Climbing*, *Simulated Annealing*, and *Tabu Search*) progressively refine the layout through neighbourhood exploration. *Simulated Annealing* excels in complex search spaces where probabilistic acceptance of worsening moves enables exploration beyond local optima. *Tabu Search* consistently achieves the highest scores across most datasets, effectively escaping local traps through memory-guided diversification. Both methods demonstrate robust performance, balancing runtime, memory, and solution quality.

The *Genetic Algorithm* introduces a population-based perspective, combining specialised crossover, guided mutation, and local refinement to explore diverse layouts. Despite its potential to reach high peak scores, the algorithm often fails to converge within practical runtime limits, and its stochastic variability produces inconsistent results. Consequently, for the City Plan problem under realistic constraints, Genetic Algorithm is largely impractical, although it demonstrates the theoretical advantages of population-based exploration.

Overall, the results highlight several key insights:

- Dataset topology and service complexity often dominate achievable scores, outweighing the raw capabilities of algorithms.
- More placements do not necessarily translate to higher scores; efficiency arises from strategically placing projects considering constraints.
- Trajectory-based methods (*Tabu Search* and *Simulated Annealing*) consistently achieve the best balance between solution quality, runtime, and memory usage.
- Greedy is ideal for rapid, resource-constrained solutions, while Genetic Algorithm is limited by runtime and convergence issues.

In addition, the predictive component of the project demonstrated that dataset characteristics can be used to anticipate algorithm performance before running the full optimization process. By training a Random Forest model on structural features of the instances, it was possible to estimate the expected score of each algorithm with reasonable accuracy. Although exploratory, this approach shows that machine learning models can complement empirical evaluation by providing early guidance on algorithm selection for unseen datasets.

In conclusion, the framework successfully demonstrates how a combination of constructive heuristics, local search, and population-based strategies can explore the large and highly constrained City Plan search space. For practical applications where maximizing solution quality is the priority, *Tabu Search*

and *Simulated Annealing* are recommended, whereas *Greedy* serves as a dependable choice under strict computational or temporal limitations.

These results also provide predictive guidance: datasets with fewer projects and service types can achieve high scores with minimal placements, whereas highly constrained datasets require intensive exploration, validating the importance of algorithm selection based on problem characteristics.

REFERENCES

- [1] A. A. Reihani, "Comparison of Optimisation Methods for The Knapsack Problem: Steepest Ascent Hill Climbing vs. Simulated Annealing," Dept. Comput. Sci., Univ. York, U.K., 2023.
- [2] Z. Yang, G. Wang, and F. Chu, "An effective GRASP and tabu search for the 0–1 quadratic knapsack problem," *Computers & Operations Research*, vol. 40, no. 5, pp. 1176–1185, 2013.
- [3] A. Rezoug, M. Bader-El-Den, and D. Boughaci, "Guided genetic algorithm for the multidimensional knapsack problem," *Memetic Computing*, vol. 10, no. 1, pp. 29–40, 2018.
- [4] K. Tole, R. Moqa, J. Zheng, and K. He, "A Simulated Annealing approach for the Circle Bin Packing Problem with Rectangular Items," *Computers & Industrial Engineering*, vol. 176, Art. no. 109004, 2023.
- [5] K. Hrytsyuk, "CityPlan: Final task for Hash Code 2018," GitHub repository, 2018. [Online]. Available: <https://github.com/kostyaHrytsyuk/CityPlan>. [Accessed: Mar. 1, 2026].